AI - Assignment 2

Conceptual Summary

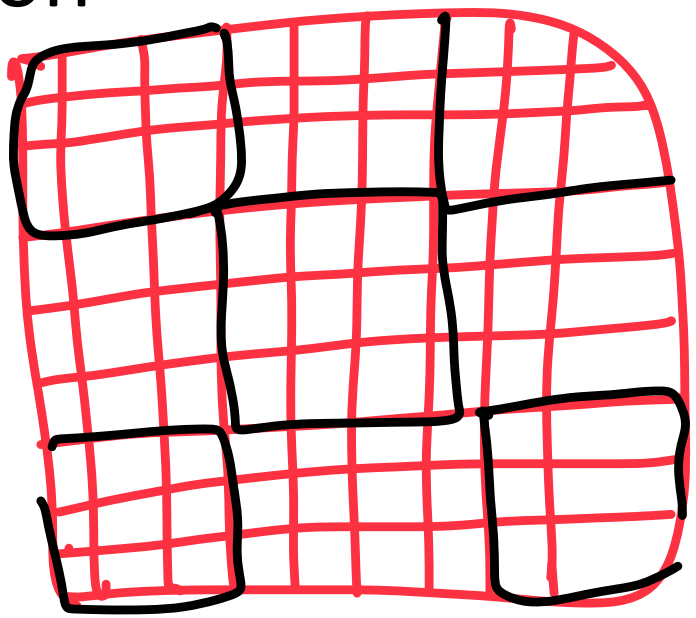
By - Manvendra

Q1

Sudoku Solver Using CSP Techniques

🎯 What is the Sudoku Problem?

- 9×9 grid puzzle with 3×3 inner boxes
- Each row, column, and box must have digits 1–9 with no repetition



What is CSP in Sudoku?

Constraint Satisfaction problem

- CSP = Variables, Domains, and Constraints
- Variables: 81 cells; Domain: {1–9}; Constraints: Unique values in units
- CSP gives a structured, efficient way to solve Sudoku

3 main Constraint in Sudoku

Unique no in
→ Row
→ Column
→ 3x3 box

🔄 CSP ~~Techniques~~ Methods in Sudoku

- Backtracking Search: Try & backtrack on failure
- Forward Checking: Prune domains after each assignment
- Arc Consistency (AC-3): Enforce constraint propagation preemptively
- Heuristics: Smart choice of variables and values

Backtracking Search

- Recursive trial-and-error approach
- Tries digits one by one in each cell
- Backtracks if a conflict is detected

Forward Checking

- After assigning a digit, remove it from peers' domains
- Avoids exploring doomed branches
- Speeds up solving by early pruning

Arc Consistency (AC-3)

- Ensures variable pairs have consistent values
- Removes values that would cause future conflicts
- Reduces problem size before actual search

Heuristics in CSP

- MRV: Pick the cell with fewest legal values
- Degree Heuristic: Prefer cell with most constraints
- LCV: Pick digit that limits others the least

How These Techniques Improve Time

- Forward Checking prevents invalid paths early
- AC-3 prunes deeply before starting the search
- Heuristics lead to smarter decisions, fewer mistakes
- All reduce total backtracks and runtime

🔍 Why These Techniques Improve Performance

- Forward Checking avoids wasted effort by early pruning
- Arc Consistency detects conflicts before they even occur
- Heuristics prioritize hard cells and safe digits
- Together, they reduce search space, time, and failure rate

✅ Final Conclusion

- Sudoku as CSP is powerful when paired with smart strategies
- Constraint propagation (FC, AC-3) + smart heuristics = big gains
- Backtracking is a base; CSP makes it intelligent and fast

🎓 Summary

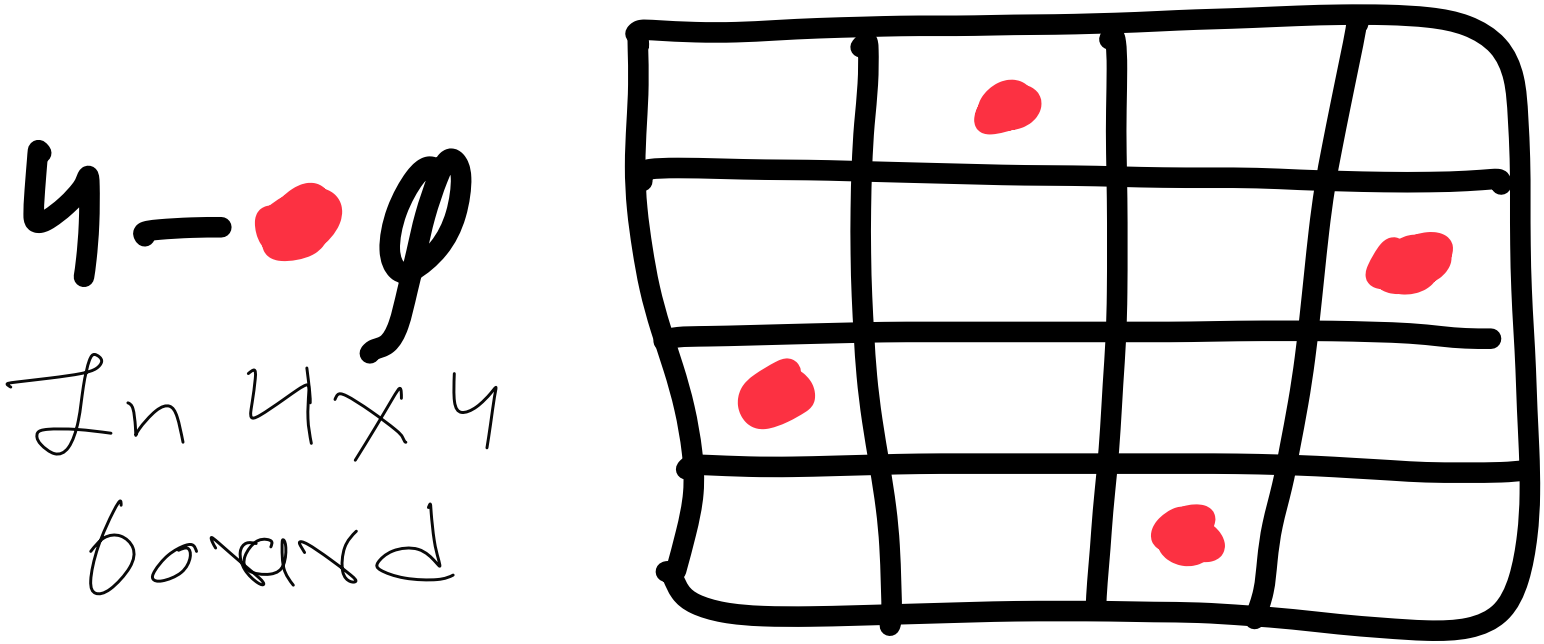
- Sudoku isn’t just logic—it’s a structured CSP problem
- CSP techniques turn brute-force into brilliance
- Now you know how to solve it like a pro!

Q 2

Simulated Annealing & Hill Climbing for N-Queens

🎯 Problem Statement

- Place N queens on an N×N chessboard
- No two queens share the same row, column, or diagonal



🌡️ What is Simulated Annealing?

- Inspired by metal cooling and solidifying
- Allows exploration by accepting worse moves
- Gradually becomes greedy as temperature decreases

SA ⇒ iron + impurities like carbon, magnesium.... make steal by melting & calling method

SA Key Concepts

- State: N-length list where index = column, value = row
- Cost Function: Counts attacking queen pairs
- Temperature (T): Controls randomness of move acceptance
- Cooling Rate: T *= cooling_rate each step

SA Parameters & Ranges

- Initial Temperature: 10 – 50
- Cooling Rate: 0.85 – 0.99
- Max Steps: 50 – 1000
- Lower T = greedier, Higher T = more exploration

15 is best for 8x8

🛠️ SA Enhancements

- Smart Initialization
- Visual Chessboard Output with Actions
- Best State Memory
- Adaptive Restarts
- Temperature Reset on Restart

🔁 When T = 0 in SA...

- It becomes pure Hill Climbing (HC)
- No randomness: always takes best local move
- Can get stuck in local minima (no worse moves accepted)

Hill Climbing Basics

- Greedy algorithm: only takes better neighbors
- Fast, simple, deterministic
- Fails if it gets stuck in a local minimum

Not always provide optimality

⚖️ Hill Climbing vs Simulated Annealing

- HC is faster but more prone to getting stuck
- SA is slower but more robust with randomness
- HC = no exploration | SA = controlled exploration
- Use HC for clean landscapes, SA for complex ones

SA can also failed with iteration reached max – iteration provided

🔧 Improving Hill Climbing

- Random Restarts to escape local minima
- Track best overall state across restarts
- Visual output for better understanding

but it still not guaranty to reach the optimality but it will reach best local min.

HC	SA
fast	Slow
stuck easily	rarely
only best move	can move worse
No Gaurantty	most Gaurantty

🔮 Final Thoughts

- "Don’t rush the process. Explore. Make mistakes. Settle into greatness."
- Simulated Annealing teaches us patience and exploration.
- Hill Climbing shows us speed and simplicity.
- Both are tools — the best choice depends on the problem.

🎓 Summary

- Simulated Annealing: versatile, powerful, good for complex landscapes
- Hill Climbing: fast and efficient, but limited
- You now understand both — use them wisely!